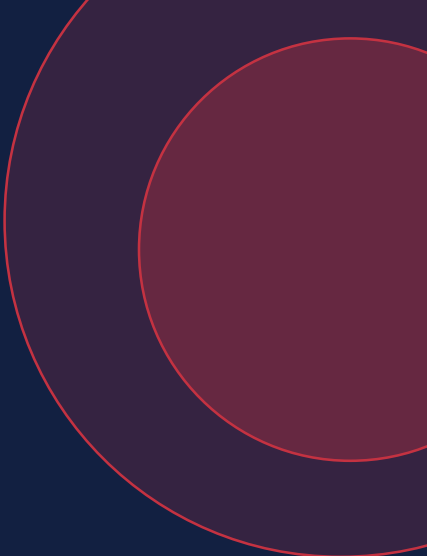




# EPIDEMIC OUTBREAK DETECTOR

A real-time epidemic surveillance & analysis system  
built with advanced data structures in C++

Mpox Dataset  
20 Regions · 30 Days

# Project Description

The Epidemic Outbreak Detector is a C++ desktop application that loads real Monkeypox (Mpox) case data, analyzes disease spread across 20 global regions over 30 days, and classifies each region's risk status in real time. The system features a full SFML graphical dashboard with 10 interactive screens backed by three custom-built data structures.

## Load & Parse

Reads a CSV dataset of daily case counts and population figures for 20 countries.

## Analyze

Computes average growth, cases per million, and sliding-window trend for every region.

## Visualize

Renders scatter plots, bar charts, line graphs, and a KD-tree diagram in a live SFML window.

# Real-World Applications

## Public Health Agencies

WHO, CDC, and national health ministries can use outbreak detectors to trigger early response protocols before a disease escalates to epidemic or pandemic level.

## Epidemic Early Warning

Automated surveillance systems flag rising trends across multiple regions simultaneously, giving governments critical lead time to allocate resources.

## Epidemiological Research

Researchers can query spatial proximity of similar outbreak profiles using KD-tree nearest-neighbor search to identify correlated transmission patterns.

## Regional Risk Profiling

Ranking regions by growth rate and cases per million helps prioritize vaccine distribution, quarantine zones, and medical supply chains.

# Dataset Overview

20

Global Regions

30

Days of Data

4

CSV Columns

600+

Entries Tracked

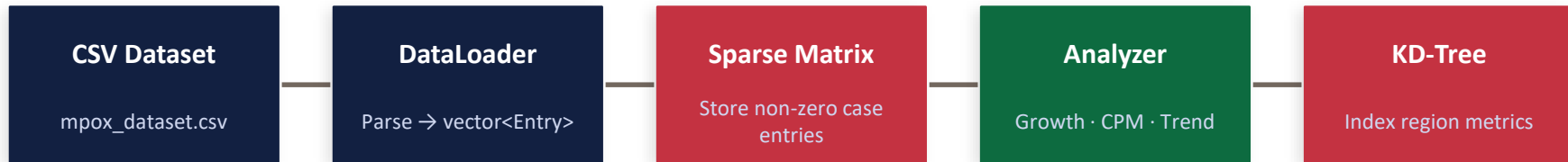
## CSV Structure

| Column     | Type    | Description               |
|------------|---------|---------------------------|
| Region     | string  | Country / region name     |
| Day        | integer | Day number (1–30)         |
| Cases      | integer | New confirmed cases       |
| Population | integer | Total regional population |

## Regions Covered

- Pakistan
- India
- United States
- United Kingdom
- France
- Germany
- Brazil
- Nigeria
- DRC
- South Africa
- Australia
- Japan
- Indonesia
- Turkey
- Saudi Arabia
- UAE
- Mexico
- Canada
- Italy
- Argentina

# System Architecture



---

## SFML Dashboard — 10 Interactive Screens

All Regions

Sparse Matrix

Sparse Operations

Analysis Report

Manage Region

Range Search

Nearest Region

Top Risk

Trend Graph

KD-Tree View

# Data Structure 1 — Sparse Matrix

## What & Why

A Sparse Matrix stores only non-zero values, making it ideal for epidemic data where most regions report zero cases on most days. Instead of a dense 20×30 grid of mostly-zero integers, only actual case events are stored as (region, day, cases) triples — saving memory and speeding up iteration.

## Operations Implemented

- `insert(region, day, cases)`
- `get(region, day) → int`
- `search(region, day) → bool`
- `update(region, day, newCases)`
- `remove(region, day)`
- `extractRegion(region, totalDays)`
- `extractNonZeroCases(region)`
- `print()` — console dump

```
struct SparseEntry {
    string  region;
    int     day;
    int     cases;
};

class SparseMatrix {
    SparseEntry entries[1000];
    int count;

    void insert(...);
    int  get(...);
    bool search(...);
    bool update(...);
    bool remove(...);
    vector<int> extractRegion(...);
};
```

# Data Structure 2 — KD-Tree (2D)

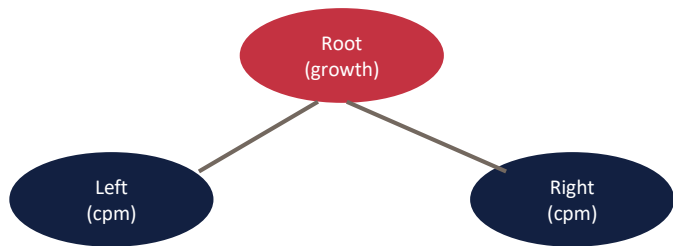
## What & Why

A K-Dimensional Tree partitions a 2D metric space — here (Growth Rate, Cases per Million) — so that spatial queries run in  $O(\log n)$  average time. After each region is analyzed, its metrics are inserted as a point in the KD-tree. This enables fast nearest-neighbor lookup, range queries, and min/max extraction.

## Operations Implemented

- `insert(region, growth, cpm)`
- `searchRegion(region) → KDNODE*`
- `nearestNeighbor(growth, cpm)`
- `rangeSearch(minG, maxG, minC, maxC)`
- `findMin(dimension) → KDNODE*`
- `findMaxGrowth() → KDNODE*`
- `getTopNByGrowth(n) → vector`
- `updateRegion(region, newG, newC)`
- `deleteNode(region)`

```
struct KDNODE {  
    string region;  
    float growth; // dim 0  
    float cpm;    // dim 1  
    KDNODE *left, *right;  
};
```



*Alternates split dimension  
at each tree level*

# Decision Tree (Analyzer)

## What & Why

The Analyzer class implements a rule-based Decision Tree that classifies each region as **SAFE**, **WARNING**, or **OUTBREAK** based on three computed metrics:

- Average Growth Rate (log-scale % per day)
- Cases Per Million (7-day rolling window)
- Rising Trend (sliding-window majority check)

**OUTBREAK**

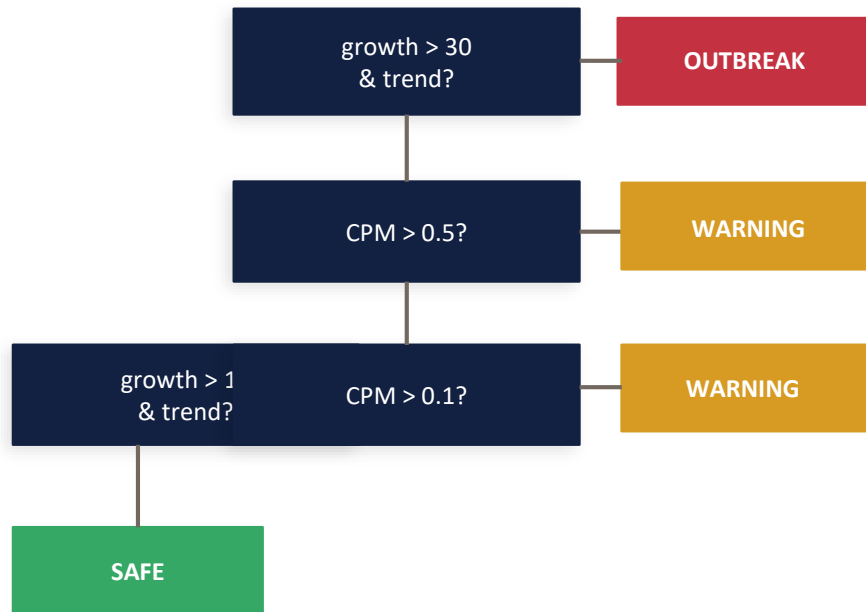
growth > 30% & rising OR  
CPM > 0.5

**WARNING**

growth > 15% & rising OR  
CPM > 0.1

**SAFE**

All other cases



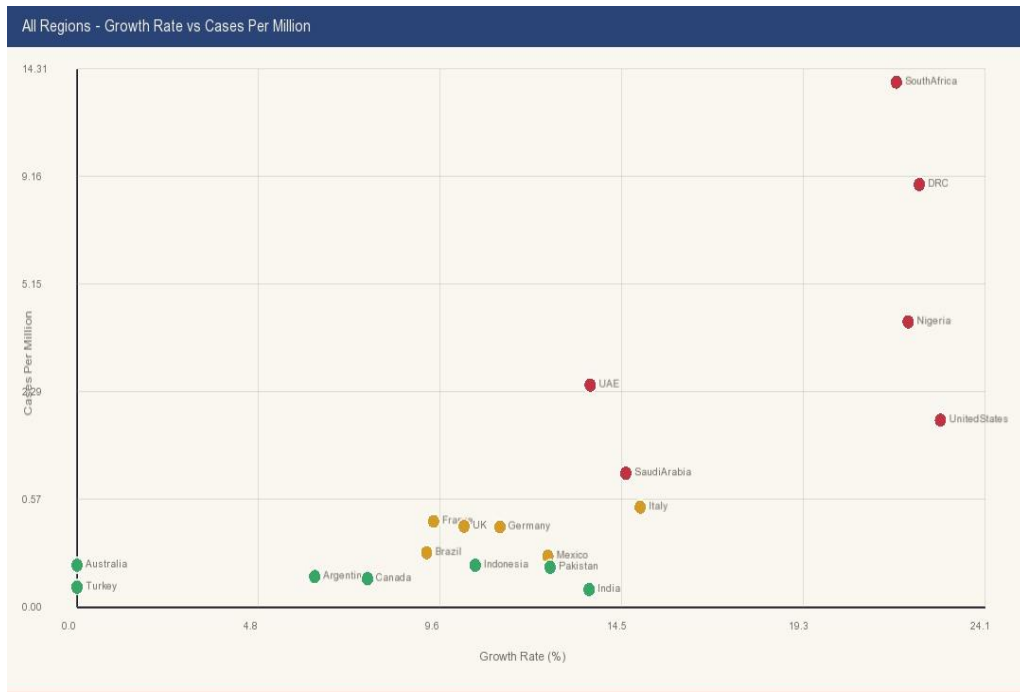


# Screen 1 — All Regions Dashboard

The default landing screen. Displays a scatter plot of all regions plotted by Growth Rate (X-axis) vs Cases Per Million (Y-axis). Each dot is color-coded by status — green (SAFE), amber (WARNING), red (OUTBREAK). A legend in the output panel summarizes each region's classification.

## Key Features

- Color-coded scatter plot (SAFE / WARNING / OUTBREAK)
- Axes auto-scale to dataset range
- Region labels rendered next to each point
- Output panel lists all region statuses
- Active sidebar button highlights current screen

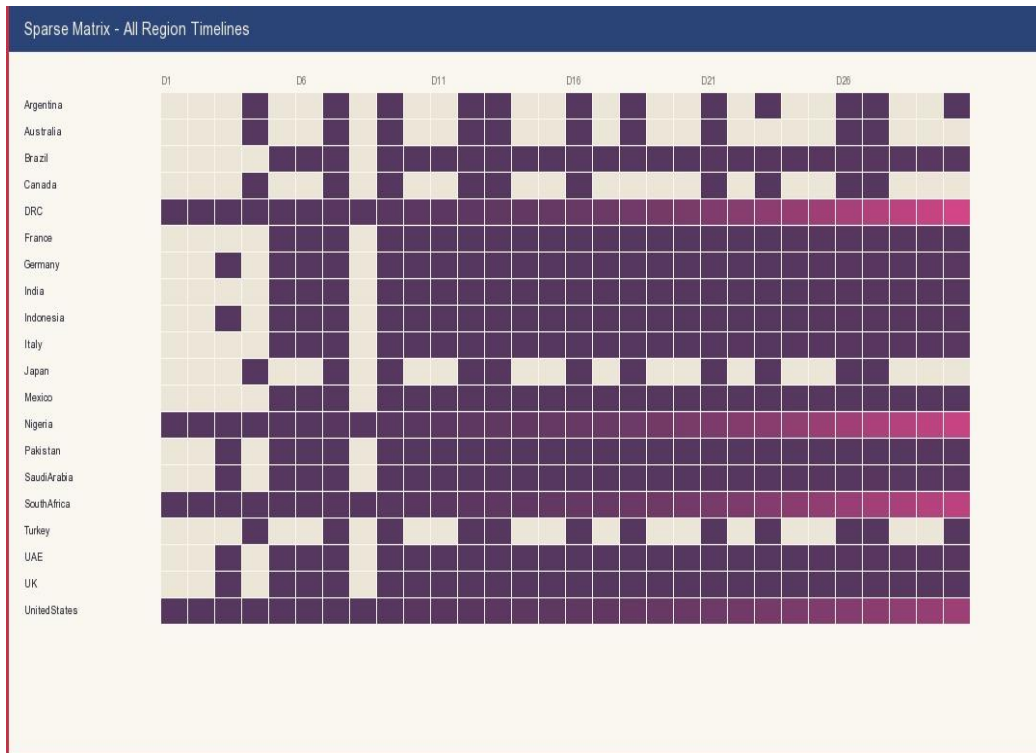


# Screen 2 — Sparse Matrix View

Renders the Sparse Matrix contents as a visual grid. Each stored non-zero entry (region, day, case count) is shown as a colored cell on the canvas. Zero-case entries are simply omitted, illustrating the memory efficiency of the structure.

## Key Features

- Rows = regions, Columns = days (1–30)
- Cell color intensity maps to case count
- Empty cells = zero — not stored at all
- Color legend shown in output panel
- Demonstrates memory savings over dense arrays



# Screen 3 — Sparse Operations (CRUD)

Exposes the full CRUD interface for the Sparse Matrix. The user can insert new entries, update existing case counts, or remove records directly from the running dataset. Changes are reflected immediately in the visual grid and downstream analytics.

INSERT

Add a new (region, day, cases) entry to the sparse matrix if not already present.

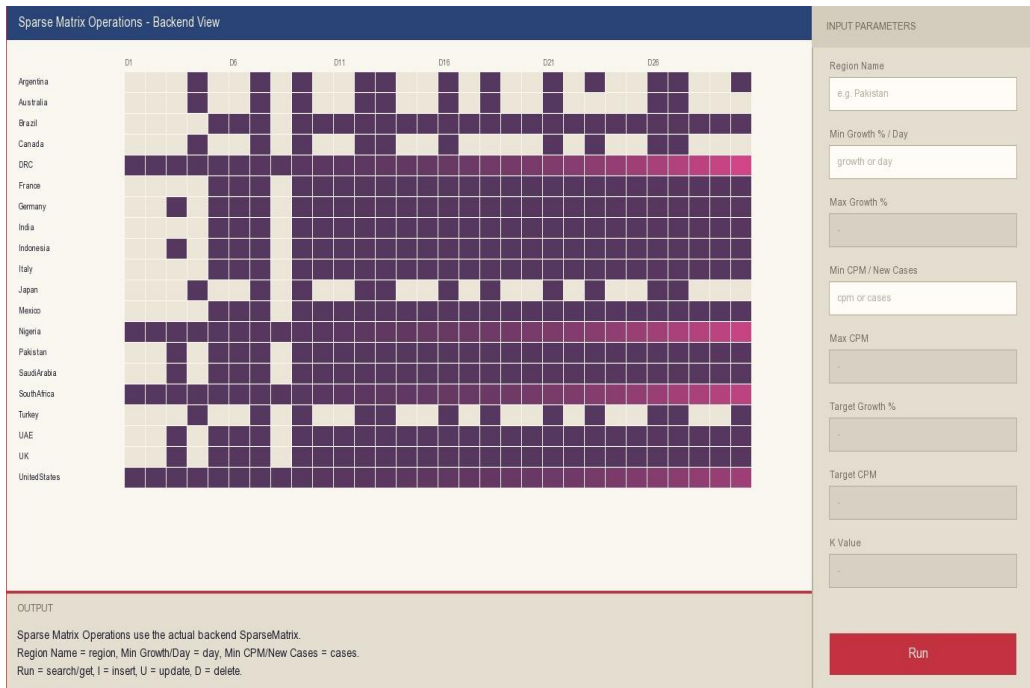
UPDATE

Change the case count for an existing (region, day) entry. Setting cases to 0 removes the entry.

DELETE

Remove a (region, day) entry. Remaining entries shift left to fill the gap.

## Sparse Operations Panel



# Screen 4 — Analysis Report

Generates a full per-region epidemic report. For every region, the system computes average daily growth, cases per million, and trend direction, then applies the Decision Tree to assign a status badge. Results are displayed as a color-coded bar chart sorted by growth rate.

## Metrics Computed Per Region

- Average Growth Rate (log-scale, % per day)
- Cases Per Million (7-day rolling average)
- Rising Trend (sliding-window majority vote)
- Final Status → SAFE / WARNING / OUTBREAK

### Analysis Report - Epidemic Status Summary

#### EPIDEMIC OUTBREAK DETECTION REPORT

##### SUMMARY STATISTICS

Total Regions Monitored: 20

Average Growth Rate: 11.42%

Highest Growth Region: UnitedStates

##### CLASSIFICATION BREAKDOWN

SAFE: 8 regions

WARNING: 6 regions

OUTBREAK: 6 regions

##### DATA STRUCTURE FLOW

Sparse Matrix stores non-zero case entries only.

Analyzer calculates growth, CPM, and trend.

Decision Tree classifies each region.

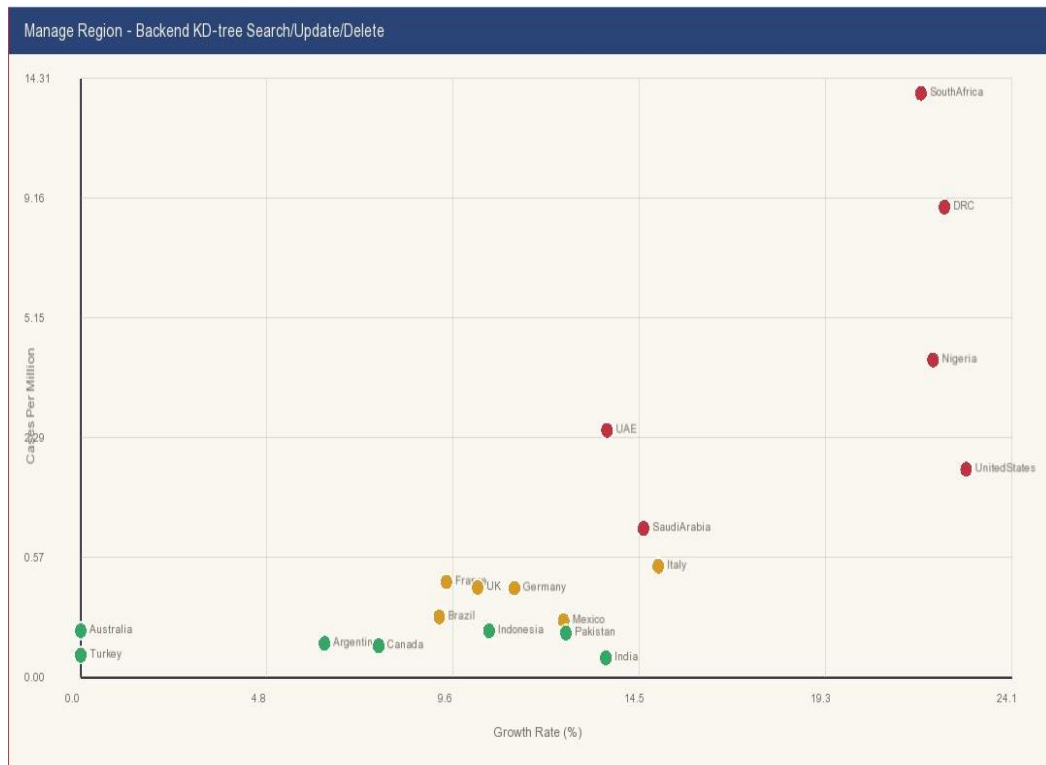
KD-tree stores analyzed points for search/range/nearest/top-K queries.

# Screen 5 — Manage Region (KD-Tree CRUD)

Allows the user to search for a region by name in the KD-Tree, update its Growth and CPM values, or delete it entirely. The scatter plot highlights the selected region. Internally, deletion is done by rebuilding the subtree — no lazy deletion.

## How KD-Tree Search Works

The search traverses the tree comparing the target region name at each node. If the region is found, a pointer to its KDNode is returned, granting  $O(\log n)$  average access. Update replaces growth/cpm in place; delete rebuilds the subtree to maintain KD-tree ordering.



# Screen 6 — Range Search

The user specifies a rectangular query window in (Growth, CPM) space.

The KD-Tree's `rangeSearch()` prunes branches early: if a subtree's bounding half-plane cannot intersect the query rectangle, it is skipped entirely. All matching region points are highlighted on the scatter plot.

## Input Parameters

- Min Growth (default 10%)
- Max Growth (default 80%)
- Min CPM (default 0.0)
- Max CPM (default 5.0)

## Why KD-Tree is efficient here:

Branch pruning avoids visiting nodes outside the rectangle, reducing comparisons from  $O(n)$  to  $O(\sqrt{n})$  on average.

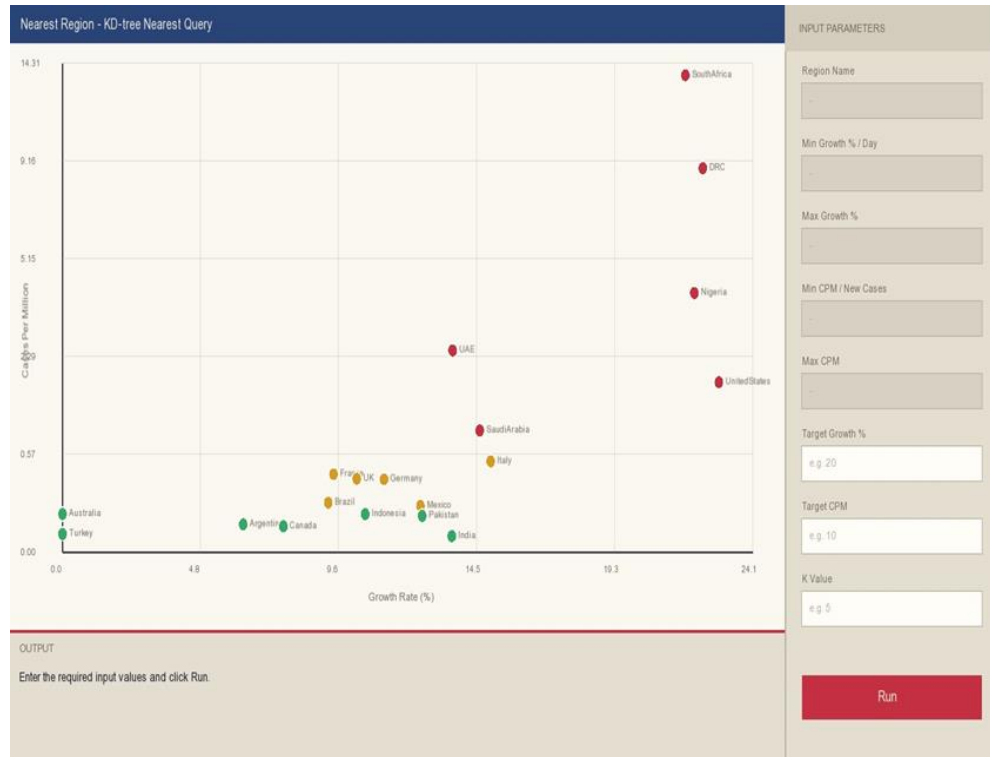


# Screen 7 — Nearest Region

The user enters a target (Growth, CPM) coordinate. The KD-Tree's nearestNeighbor() finds the single closest region using Euclidean distance in 2D, pruning subtrees when the axis-aligned distance to a node's splitting plane already exceeds the current best. The matched region is highlighted on the scatter plot.

## Algorithm Outline

1. Traverse the tree recursively
2. At each node, update bestDist if closer
3. Check if opposite subtree may contain closer point
4. Prune if axis distance  $\geq$  bestDist
5. Return bestNode at leaf



# Screen 8 — Top Risk Regions

Displays the top-N highest-growth regions. The KD-Tree collects all nodes via an in-order traversal, sorts them by growth rate, and returns the top N. The user can switch between Scatter, Line, and Bar graph modes using the input panel.

## Graph Modes

### SCATTER

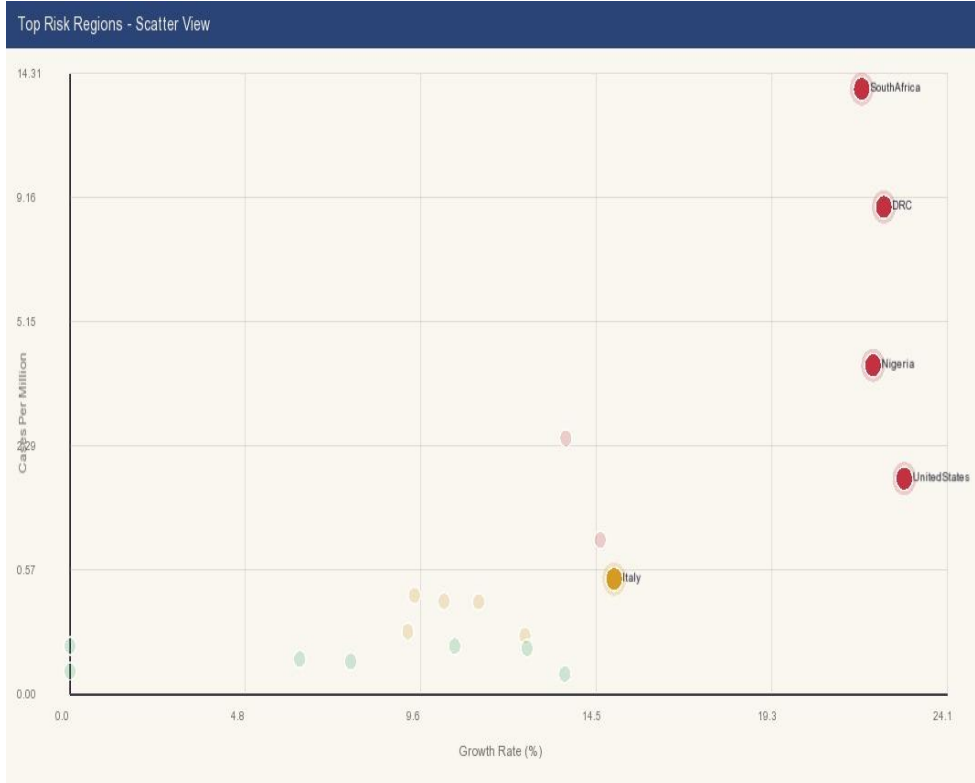
Plots top-N regions as dots in Growth-vs-CPM space, highlighted in crimson.

### LINE

Connects top-N regions by growth rank — useful to see CPM trend alongside growth.

### BAR

Horizontal bars ranked by growth rate, colored by OUTBREAK / WARNING / SAFE status.



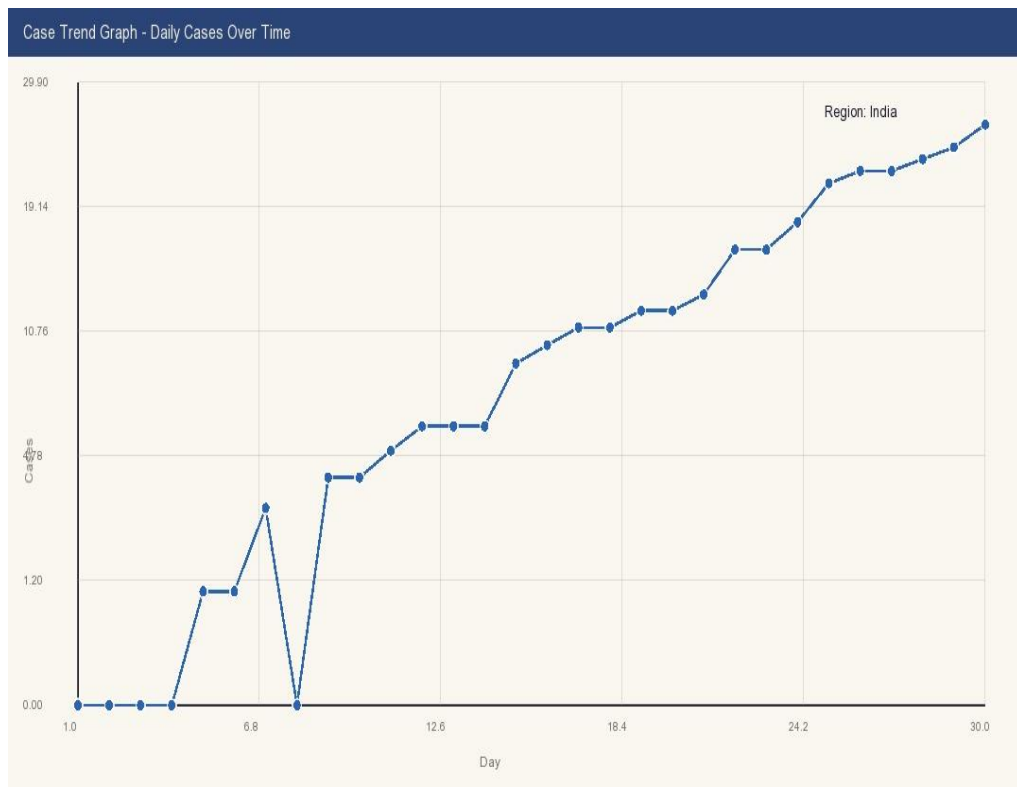


# Screen 9 — Trend Graph

Shows the day-by-day case timeline for a selected region. The user types a region name and the system extracts the full 30-day case vector from the Sparse Matrix. The line graph uses the GraphRenderer class to scale and plot each data point, making exponential growth patterns immediately visible.

## Sliding-Window Trend Detection

The Analyzer checks the last windowSize (default 5) data points. If the majority of consecutive pairs show an increase, the region is flagged as 'Rising Trend'. This is visualized as a red annotation on the trend line where the rising window begins.

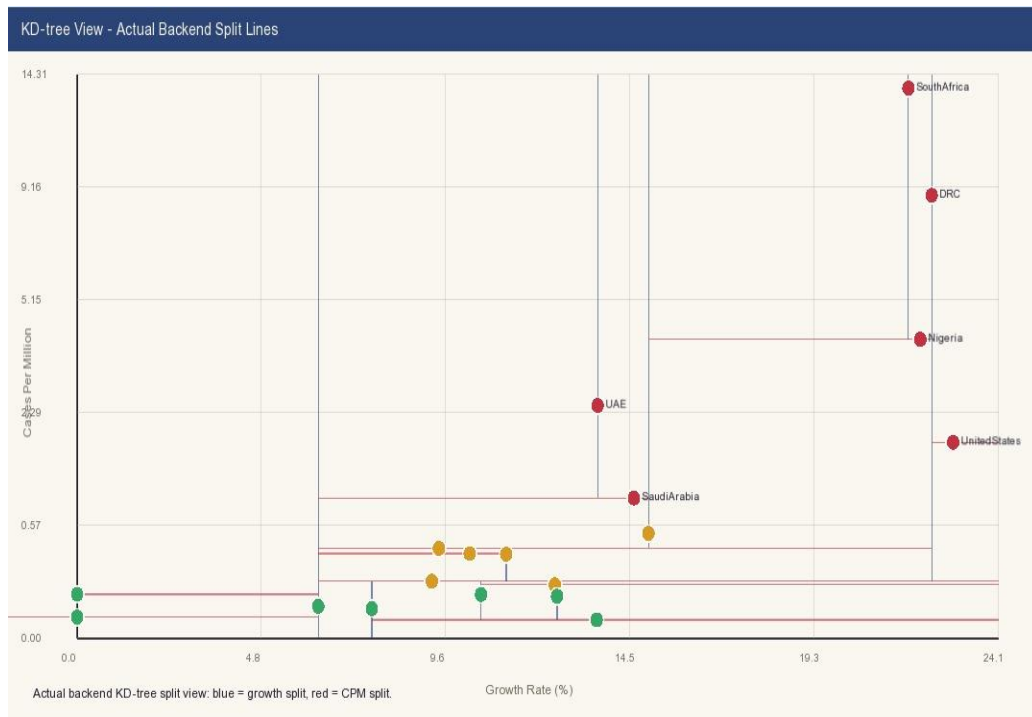


# Screen 10 — KD-Tree Visualizer

Renders the KD-Tree as an actual tree diagram using SFML. Each node is drawn as a labeled box containing the region name, growth rate, and CPM value. Edges connect parent to child nodes with color-coded depth levels so the alternating split dimension is visually apparent.

## Visualization Details

- Depth 0 (root): splits on Growth (dim 0) — crimson
- Depth 1: splits on CPM (dim 1) — navy
- Depth 2+: repeats, colors cycle
- Each node label: Region | G: growth | C: cpm
- Tree auto-scales and centers on the canvas



# Key Takeaways

---

## **Sparse Matrix**

Stores only non-zero case events — memory-efficient and faster to iterate than a full dense grid.

## **KD-Tree (2D)**

Enables  $O(\log n)$  nearest-neighbor, range search, and min/max queries across region metrics.

## **Decision Tree**

Rule-based classification assigns SAFE / WARNING / OUTBREAK status to each region automatically.

## **SFML Dashboard**

10 interactive screens deliver real-time visualization of all three data structures simultaneously.